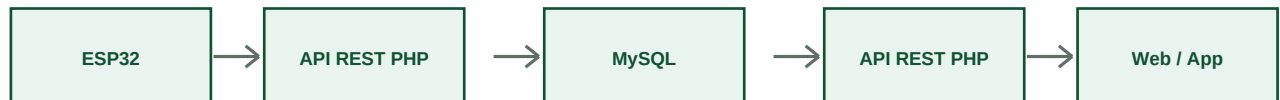


Manual de uso del sistema

Guia para levantar la base de datos, abrir la web/app, usar ngrok, correr pruebas y entender la arquitectura.

Todo pasa por HTTP JSON; nadie se conecta directo a MySQL excepto la API.



Documento de apoyo para montaje, configuracion y uso del proyecto. Usa rutas y archivos reales del repositorio para que el equipo pueda seguirlo sin adivinar.

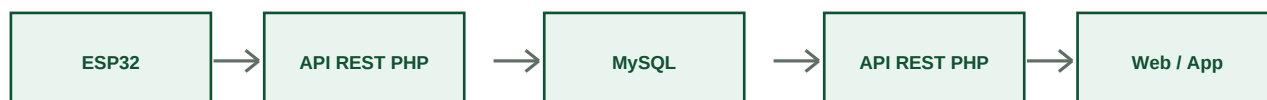
1. Que es el sistema

El proyecto es un invernadero inteligente con backend PHP, base de datos MySQL, panel web, app Android, simulador ESP32 y firmware base para Arduino IDE.

La regla principal de arquitectura es simple: los dispositivos no hablan directo con MySQL. Todo pasa por la API REST PHP.

2. Arquitectura general

Todo pasa por HTTP JSON; nadie se conecta directo a MySQL excepto la API.



- ESP32: lee sensores, controla salidas y manda JSON a la API.
- API REST PHP: valida datos, guarda en MySQL y responde a web/app/ESP32.
- MySQL: guarda lecturas, estados, configuracion, eventos, accesos RFID, comandos y cola de automatizacion.
- Web/App: visualizan datos y crean comandos permitidos para actuadores.

3. Requisitos de la computadora

- Docker Desktop instalado y abierto.
- Git para clonar o actualizar el repositorio.
- PowerShell para correr pruebas.
- Navegador web.
- ngrok si se quiere conectar el ESP32 o app desde una red externa.
- Arduino IDE si se cargara firmware al ESP32.

4. Levantar el sistema desde cero

- 1 Abrir PowerShell en la raiz del repositorio.
- 2 Si aun no existe `.env`, ejecutar `copy .env.example .env`.
- 3 Ejecutar `docker compose up -d --build`.
- 4 Esperar a que los contenedores `app`, `db` y `phpmyadmin` esten listos.
- 5 Abrir `http://localhost:8080/api/status.php` y confirmar que la API responde.
- 6 Abrir `http://localhost:8080/web/` para usar el panel web.

Comandos utiles

```
docker compose up -d --build
docker compose down
```

```
# Reinicio total de base de datos: borra el volumen MySQL
docker compose down -v
docker compose up -d --build
```

5. URLs importantes

Servicio	URL	Uso
Panel web	http://localhost:8080/web/	Pantalla principal del sistema
API status	http://localhost:8080/api/status.php	Verificar que API y BD funcionan
phpMyAdmin	http://localhost:8081	Consultar base de datos manualmente
API base	http://localhost:8080/api	URL local para endpoints

6. Credenciales de phpMyAdmin

Campo	Valor
Servidor	db
Usuario	invernadero_user
Contraseña	invernadero_pass
Base de datos	invernadero_iot

7. Apartados del panel web

Apartado	Que muestra / para que sirve
Estado de conexion	Indica si la API y la base de datos estan disponibles.
Ultima lectura de sensores	Temperatura, humedad ambiental, humedad de suelo, valor raw del sensor y luz en lux.
Ultimo estado de actuadores	Ventilador, bomba, lampara y servo. Muestra turno: libre, usuario o automatizacion.
Comandos pendientes	Indica si hay comandos manuales esperando procesamiento.
Configuracion de automatizacion	Permite revisar y editar umbrales de temperatura, humedad, luz y duracion de riego.
Ultimas lecturas	Historial reciente de sensores guardado en MySQL.
Accesos RFID recientes	UID, usuario, autorizado/rechazado y servo abierto/cerrado.
Eventos de actuadores	Cambios de estado y motivo: automatizacion, comando manual, RFID, cola, etc.
Comandos recientes	Comandos creados desde web/app y su estado: pendiente, ejecutado, fallido o cancelado.

8. Botones y controles principales

- Cards de actuadores: si el actuador esta `Apagado - Libre`, se puede tocar para encenderlo bajo control usuario.
- Card con `Encendido - Control usuario`: se puede tocar para apagar y liberar el actuador.
- Card con `Encendido - Control automatizacion`: queda bloqueada para que el usuario no interrumpa la automatizacion.
- Cards de configuracion: al tocarlas permiten cambiar umbrales o activar/desactivar automatizaciones.
- La app Android usa la misma idea: cards por actuador, actualizacion automatica y URL de API configurable.

9. Turnos por actuador y cola de automatizacion

Cada actuador controlable tiene control actual: `libre`, `usuario` o `automatizacion`. Aplica a ventilador, bomba y lampara. El servo queda fuera porque pertenece al flujo RFID.

- Usuario toma control solo si el actuador esta libre y apagado.
- Automatizacion toma control solo si el actuador esta libre y apagado.
- Si automatizacion necesita un actuador ocupado por usuario, crea una tarea en `cola\_automatizacion`.
- Cuando el usuario libera, la cola se ejecuta si la condicion sigue activa o se cancela si ya no aplica.
- No se duplican tareas pendientes para el mismo actuador y accion.

10. Base de datos: tablas principales

Tabla	Contenido
lecturas	Historico de sensores.
configuracion_automatizacion	Umbrales y banderas de automatizacion.
estados_actuadores	Estados y control actual por actuador.
cola_automatizacion	Tareas automaticas pendientes, ejecutadas o canceladas.
comandos_actuadores	Comandos creados por web/app para ventilador, bomba o lampara.
eventos_actuadores	Cambios y motivos de actuadores.
tarjetas_rfid / accesos_rfid	Tarjetas demo y registros de acceso.
calibraciones_sensores	Datos de calibracion disponibles para sensores.

11. Endpoints mas usados

Endpoint	Uso
GET /api/status.php	Verificar API y conexion a BD.
GET/POST /api/lecturas.php	Consultar o guardar lecturas de sensores.
GET/POST /api/actuadores.php	Consultar o guardar estado/control de actuadores.
GET/PUT /api/configuracion.php	Consultar o cambiar configuracion de automatizacion.
GET/POST/PUT /api/comandos.php	Crear, listar o cerrar comandos manuales.
GET/POST/PUT /api/cola_automatizacion.php	Consultar o administrar cola automatica.
GET/POST /api/eventos.php	Consultar o registrar eventos de actuadores.
GET/POST /api/accesos.php	Consultar o registrar accesos RFID.

12. Usar ngrok

- 1 Levantar Docker primero.
- 2 Ejecutar `ngrok http 8080`.

- 3 Copiar la URL HTTPS que entrega ngrok.
- 4 Para el ESP32, poner esa URL en `API\_BASE\_URL` terminando en `/api`.
- 5 Para la app Android, escribir la URL en el campo de API base.
- 6 Recordar que la URL gratis de ngrok puede cambiar al reiniciar ngrok.

```
ngrok http 8080

# Ejemplo resultado:
# https://ejemplo.ngrok-free.app

# API base para ESP32/app:
# https://ejemplo.ngrok-free.app/api
```

13. Pruebas disponibles en el repositorio

Las pruebas estan en la carpeta `tests/`. Algunas reconstruyen la base desde cero y borran el volumen de MySQL. Revisar el nombre antes de correrlas.

```
powershell -ExecutionPolicy Bypass -File .\tests\probar_fase8_1_app_actualizacion.ps1
powershell -ExecutionPolicy Bypass -File .\tests\probar_fase9_automatizacion.ps1
powershell -ExecutionPolicy Bypass -File .\tests\probar_fase11_bloqueos.ps1

# Simulador ESP32:
powershell -ExecutionPolicy Bypass -File .\tests\simular_esp32.ps1
powershell -ExecutionPolicy Bypass -File .\tests\simular_esp32.ps1 -ModoContinuo
```

14. Que hay en el repositorio

Ruta	Contenido
api/	Endpoints PHP y helpers de conexion/validacion.
sql/init.sql	Estructura inicial de MySQL y datos semilla.
web/	Panel HTML/CSS/JS.
android-app/	App Android Java/XML.
esp32/firmware_base/	Firmware Arduino y plantilla config.h.
tests/	Pruebas PowerShell y simulador ESP32.
docs/	Guías auxiliares de ejecucion, ngrok, entrega y modelo de BD.
Dockerfile / docker-compose.yml	Contenedores de app, MySQL y phpMyAdmin.

15. Solucion rapida de problemas

Problema	Revision
API no responde	Confirmar Docker Desktop abierto, `docker compose ps` y puerto 8080 libre.
phpMyAdmin no entra	Usar servidor `db`, no `localhost`.
ESP32 no conecta	Revisar SSID, contraseña, URL API y que la PC/ESP32 esten en la misma red.
ngrok cambia URL	Actualizar `API_BASE_URL` del ESP32 o campo de API en app.
Actuador no responde	Revisar control actual: automatizacion y usuario no se interrumpen.
Relay funciona al reves	Cambiar `RELAY_ON_LEVEL` y `RELAY_OFF_LEVEL`.